多仪协同智能体

摘要

针对电子实验中仪器分散操作、过程记录繁琐和指导不及时的问题，我们设计并实现了**多仪协同智能体**。系统基于 **ZedBoard 开发板**与 **AMD Zynq-7020** 器件，将直流电源、信号发生器和示波器接入同一平台，结合**语音、视觉与大模型服务**，完成实验指导、仪器操作、状态核对和结果整理。

系统以板端 Linux 应用为核心，通过 USB 和以太网连接三台仪器，在本地完成视觉的**学生身份识别**和**举手判定**。学生观察显示界面并操作板卡进行**页面交互**。语音由本地模型或 Hy 云端转写，用于操作仪器或询问问题；DeepSeek 结合当前步骤和仪器状态生成操作方案、指导与总结。板卡经串口连接电脑，由电脑访问云端并回传结果。导入指导书后，DS 生成实验步骤与接线指导图。离线时，**本地语音识别与指令解析**支持已实现的明确仪器指令。

仪器操作由板端程序检查设备、通道和参数，执行后读取实际状态。仪器操作按设备时序执行，并通过参数回读核对结果。控制结束后触发三台仪器采样，新读数及时显示并突出变化，整轮完成后**统一更新刷新时间**。操作与测量回执按实验编号保存，实验总结引用本轮已确认的结果。

针对板卡计算与内存资源有限的条件，本地语音服务**复用已加载模型**，视觉采用**快速筛选与较高分辨率复核**；普通语音期间暂停视觉，指导书解析期间释放模型，任务结束后按条件恢复。仪器状态由后台集中采集、页面共享，使语音、视觉、显示与仪器服务能够协调运行。

验证结合历史结果复算、离线回放、QEMU 模拟对比和板卡真人联调。本地模型在 415 条合成语音测试记录中，完整录音直接输入的整句正确率为 **99.28%**；工作站 **1654 次回放仅加载模型一次**，识别与指令解析结果均与基线一致。在 QEMU Cortex-A9 中，同时调整视觉运行库与计算分辨率后，推理中位耗时由 **3607.67ms 降至 786.47ms**。板卡联调验证了语音操作、仪器回读、举手求助与实验总结的衔接。系统为既有仪器增加了统一交互、步骤指导与过程记录能力，可用于电子实验教学和竞赛训练。

第一部分 项目概述

1.1系统功能与应用价值

多仪协同智能体将直流电源、信号发生器和示波器组织为一个可对话、可回读、可记录的实验工作台。系统集中呈现三台仪器状态，支持按设备与通道进行参数设置和测量查询，并将执行结果同步到页面与实验记录。语音操作让学生在接线、持探头和观察波形时直接发出指令，减少在多个仪器菜单之间切换。

系统围绕实验进度提供连续支持：上传Word或PDF指导书后生成步骤与接线指导图；结合学生身份、当前步骤和仪器状态回答问题；学生举手时提供当前步骤的帮助；实验完成后汇总本轮操作与测量结果。仪器控制、教学内容与结果记录集中在同一工作区，适用于电子类课程实验、竞赛训练和小型研发实验台。

1.2选题背景与技术基础

传统电子实验台的测量能力较强，但指导书、仪器面板和实验报告通常相互独立，学生需要自行完成步骤理解、设备配置、读数抄录和结果整理。多名学生同时实验时，重复操作说明与求助也占用教师时间。因此，本作品选择赛题指南中的自主选题初级赛道，以提升实验操作与指导效率为设计目标。

现有仪器控制软件已经为设备互联和数据管理建立了基础。NI-VISA为USB、以太网等仪器接口提供统一的通信支持[1]；Keysight BenchVue可集中管理多台仪器的数据日志，并按型号、时间等信息组织测量记录[2]。多仪协同智能体面向电子实验教学，将多仪器控制与自然语言操作、学生身份、当前步骤及测量回执结合，在ZedBoard上统一组织本地语音、视觉和实验应用，使仪器操作进一步衔接步骤指导与结果总结。

1.3主要技术特点与创新

1) 仪器操作与真实状态核对。将三类仪器接入统一适配层，执行后按设备完成状态和参数回读确认结果；控制完成触发整轮采样，兼顾新读数及时显示与三张卡片统一计时。该设计解决命令发送、仪器生效和页面反馈不同步的问题。

2) 有限资源下的多模型协同。语音模型常驻复用，视觉使用快速筛选与高分辨率复核；普通语音任务暂停视觉计算，指导书解析按内存需求卸载和恢复模型。系统据任务占用安排资源，支持语音、视觉与实验应用在同一板卡运行。

3) 实验上下文与结果记录关联。将学生身份、当前步骤和仪器状态用于求助，并为每轮实验分配编号，使操作、测量和总结保持对应；异步结果返回时检查归属，避免旧结果写入新实验。该设计把交互和记录纳入同一实验过程。

1.4主要能力与性能指标

表1-1主要能力与性能指标概览

项目	能力或结果	条件与详细说明
主控与接入	ZedBoard / Zynq-7020；三类仪器	PS侧Linux应用；USB、以太网，见第二章
实验过程支持	指导书、语音、身份与举手、总结	从开始到结束的使用流程，见2.4节
状态反馈	3s整轮调度；变化及时显示	控制完成可提前触发；共享轮次，见3.4节
语音测试	整句正确率99.28%；常驻加载1次	415条合成测试音频；1654次x86回放，见4.2节
视觉优化	模拟中位耗时降低78.2%	运行库与分辨率同时调整，见4.3节
板卡资源	本地真人联调内存溢出计数最大值0	565次采样的历史记录，见4.1节

1.5设计流程

系统设计流程如图1-1所示。先从多仪器实验的操作与指导需求确定系统边界，再分别实现仪器适配、语音转写、视觉事件和页面工作区，最后以同一实验流程将各模块联通。仪器与模型的实现细节集中在第三章，优化效果和整机成果在第四章核对。

设计迭代围绕实际故障展开：命令发送后需等待设备完成并回读；多模型共存需控制任务峰值；自动总结需核对回执归属。各次修改均保留对应条件与原始记录，再将可重复的排查方法提炼为技能。


需求分析：统一仪器操作、实验指导与过程记录
方案确定：ZedBoard本地服务 + 仪器适配 + 多模态交互
① 仪器控制设计
设备识别与协议适配
DP832 / AFG1062 / MSO22
参数、通道与回执统一
USB / USBTMC / 以太网SCPI
控制与采样互斥
顺序采集 / 共享缓存 / 变化高亮
② 软件语音交互设计
本地 / Hy语音识别
音频转换为文本
明确操作解析 / DS规划
生成设备、通道与参数
本地工具校验与执行
回读状态并形成回执
③ 视觉交互设计
摄像头采集与轻量推理
身份识别 / 姿态关键点
跨帧状态判定
连续多帧举手确认
事件关联与实验上下文
当前学生 / 实验步骤 / 求助
④ 实验应用与整机集成
指导书解析与编排
结构化步骤 / 接线提示
上下文与记录管理
身份 / 步骤 / 仪器快照
执行、反馈与记录
操作回执 / 测量回执 / 总结
⑤ 资源与时序优化
语音常驻复用
减少重复初始化
轻量视觉运行
NCNN与计算分辨率
模型生命周期调度
暂停 / 卸载 / 恢复
通信与事件同步
整轮刷新 / 切换回执
⑥ 验证与迭代
离线回放与结果复算
逐条输出 / 结果一致性
同环境模拟性能对比
QEMU耗时 / 内存
板卡真人联调
三仪器 / 语音 / 视觉 / 总结

图1-1系统设计流程

第二部分 系统组成与使用流程

2.1总体架构

系统总体架构如图2-1所示，以ZedBoard开发板上的Zynq-7020 PS侧嵌入式Linux服务为核心。XVF3800麦克风与UVC摄像头通过USB接入板卡：语音识别可选择板端本地模型或Hy云端服务，将音频转换为文本；视觉模块在板端识别学生身份和举手事件。识别结果与页面中的文字输入一起进入“实验应用与交互调度”模块，结合当前实验步骤和仪器状态，组织问答、仪器操作与记录。系统界面由板端浏览器运行，通过HDMI接口输出至显示屏，用于文字输入、实验步骤查看和仪器状态展示。Hy语音识别与DeepSeek（下文简称DS）共用电脑转发通道：板卡通过UART串口发送请求，由电脑转发至相应云端服务并回传结果。其中，Hy负责音频转写，DS负责语义理解、操作规划、实验指导与总结，具体仪器操作由板端程序检查并执行。

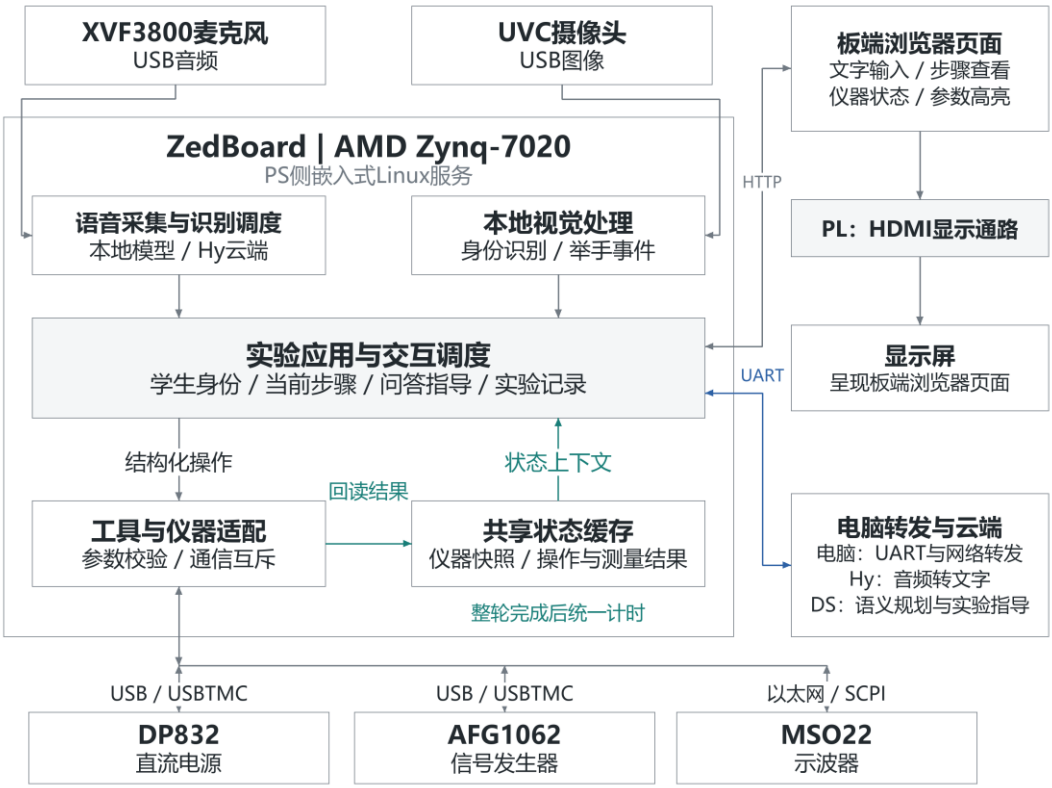


图2-1系统总体架构与主要数据流

仪器操作由图中的“工具与仪器适配”模块执行。该模块接收设备、通道、参数等操作信息，检查参数是否合法，并协调采样与控制对仪器的访问，防止同时

通信造成冲突。DP832直流电源和AFG1062信号发生器通过USB/USBTMC接入，MSO22示波器通过以太网SCPI接入。执行后读取的实际状态写入“共享状态缓存”，供实验应用和浏览器页面共同使用。控制完成后，系统触发三台仪器的整轮采样，页面统一显示整轮刷新时间并高亮变化参数，便于学生核对操作效果。

2.2软硬件划分依据

软硬件划分以外设接口、运行环境和任务特点为依据。Zynq-7020的处理系统（PS）包含ARM处理器，可运行Linux；可编程逻辑（PL）可实现专用硬件电路[3]。仪器通信、本地语音与视觉识别、页面服务和实验管理由PS上的Linux应用完成，便于共享设备状态与实验记录，通过轻量模型与任务调度控制计算和内存开销；PL沿用现有ADI显示参考设计，承担HDMI显示输出。Hy提供云端语音识别，DS负责语义理解、问答、指导编排和总结，电脑在板卡串口与云端网络服务之间转发请求和结果。

2.3硬件连接与接口

表2-1硬件组成与接口

模块	连接及信号	职责
ZedBoard开发板	Zynq-7020；SD卡启动	运行本地服务、模型与控制程序
RIGOL DP832直流电源	板卡USB主机口→仪器USB DEVICE口；USBTMC/SCPI	三通道状态与电源控制
Tektronix AFG1062信号源	板卡USB主机口→仪器USB DEVICE口；USBTMC/SCPI	双通道信号配置与回读
Tektronix MSO22示波器	以太网；TCP传输SCPI命令	波形观察、刻度与测量读取
XVF3800麦克风	USB音频输入	采集口语问题与仪器指令
Logitech UVC摄像头	USB视频	学生识别与姿态事件输入
显示器与板载按键	HDMI OUT转DVI-D；按键事件	状态展示、导航和交互
电脑与云端服务	UART转发及网络接口	转发Hy、DS请求并返回结果

主控板为ZedBoard，启动归档对应器件为xc7z020clg484-1，主板标识为Xilinx Zynq ZED。仪器网络中，板卡eth0地址为192.168.50.10/24，MSO22地址为192.168.50.20，SCPI通信端口为4000。启动脚本为eth0添加仪器子网地址，并保留已有地址与默认路由，使仪器通信与其他网络连接能够同时使用。

硬件连接如图2-2所示。USB和以太网用于发送控制命令、读取仪器状态；BNC线用于传输实际电信号。在演示实验中，AFG1062的CH1通过BNC线连接MSO22的CH1。系统配置波形参数后，由示波器观测信号并返回测量结果，形成从参数设置到测量记录的完整过程。

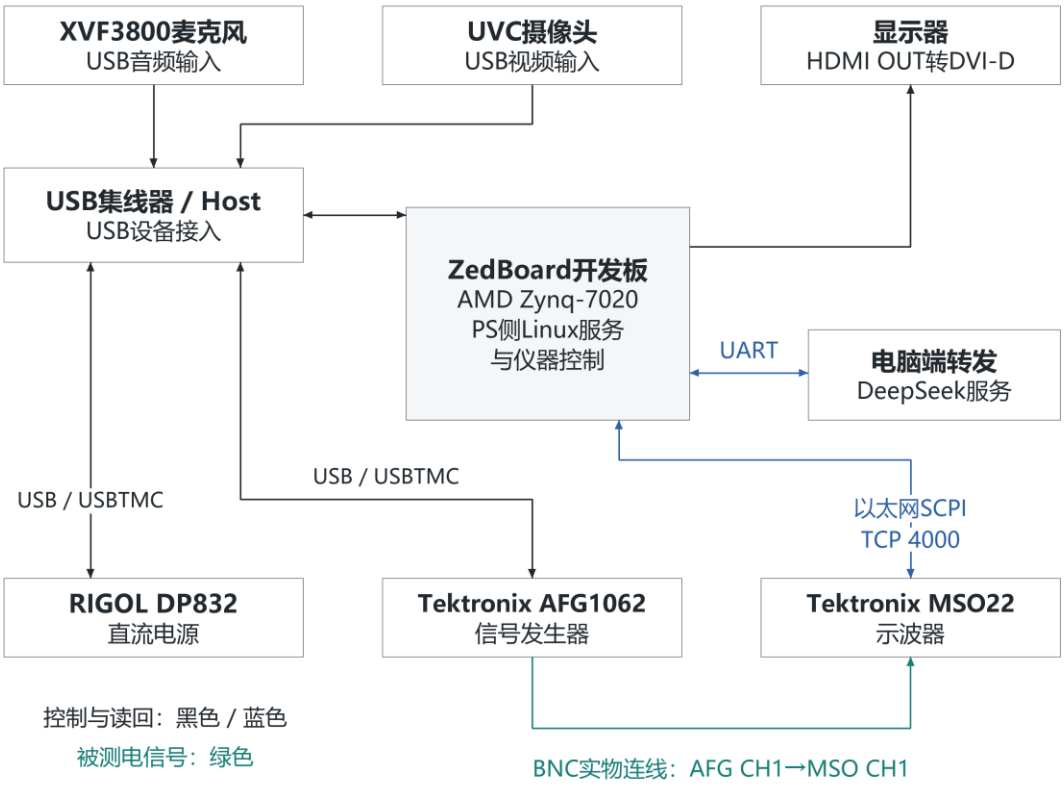


图2-2硬件接口与实验信号连接

2.4从指导书到实验总结的使用流程

以“信号源输出正弦波、示波器观察与测量”为例，系统围绕一轮实验保存指导、进度和操作结果。图 2-1 中的各入口在以下流程中协同工作。

（1）导入与准备。将实验目的、接线、1kHz/1Vpp正弦信号设定、测量及结束要求写入DOCX或带文字层的PDF，放入指导书目录并在页面选择导入。学生登记并确认身份后，系统提取正文、生成步骤和接线指导图。真实指导书

编排使用DS，语音可选择Hy或本地识别；Hy和DS须分别配置并通过电脑转发。

（2）开始与识别人物身份。开始流程后生成本轮实验编号。学生面对摄像头，如已登记则会在界面显示操作学生身份。之后按步骤核对AFG1062 CH1至MSO22 CH1的BNC接线及负载。

（3）语音操作与反馈。说“将信号源CH1设为正弦波、1kHz、1Vpp、0V偏置”， 语音转为文本后生成操作摘要，板端校验并执行，完成后对应读数高亮并更新。参数核对后，由操作者开启输出。说“把示波器时间刻度改为两百微秒每格”。更新后确认示波器刻度正确更改。任一仪器变化后，系统刷新三台仪器，整轮完成时三张卡片与底部更新同一轮次和时间。

（4）提问与举手。学生可围绕当前步骤直接语音提问，也可举手请求帮助。视觉模块将已确认身份和举手事件交给实验助手，助手结合当前步骤与仪器状态给出建议。

（5）观察与测量。按指导读取示波器CH1频率和峰峰值，核对测量项目、数值、单位及执行回执。设置为1kHz是信号源的目标参数，示波器测得的频率才是该次测量证据；两者分别记录。

（6）结束与总结。关闭信号源输出并核对关闭回执，逐项确认实验步骤完成。后台按本轮编号收集已验证操作和测量，生成实验总结；DS不可用时保留本地摘要。重新开始实验会创建新轮次并清除旧总结，后续异步响应也必须通过当前轮次与总结标识核对。

这一路径将“指导—操作—观察—求助—记录”组织为连续过程，较好地达到了我们的预期。

第三部分 软件设计与模块实现

3.1 服务启动与软件组织

软件围绕实验过程组织：页面接收学生的文字、语音和步骤确认，板端主服务统一保存实验状态、安排控制与采样，并把执行结果返回页面。摄像采集和电脑转发分别由独立服务承担，识别、控制和文档处理等耗时任务在后台执行。这样，页面能够持续显示当前进度，多个入口也能使用同一份仪器状态和实验记录。

系统由主服务启动脚本配置仪器网段并启动应用。主服务、摄像服务和云端转发入口分别使用 8765、8766 和 8767 端口；仪器网口补充 192.168.50.10/24 地址时保留已有路由。服务之间通过接口传递图像、请求与结果，仪器通信由主应用统一安排。

主服务按职责组装各模块：工作区管理指导书、学生及实验进度；语义规划模块把自然语言转换成操作计划；控制模块负责校验、执行和回读；语音与视觉模块提供交互事件；指导书处理和总结模块完成实验资料的整理。模块之间共享明确的状态，并在识别完成、控制结束或流程完成时通知后续模块，具体输入与结果见表 3-1。

语音、仪器控制和指导书处理以后台任务运行，页面取得任务标识后查询处理进度与结果；文字对话返回回答或操作卡。文字、语音和操作卡最终进入同一条控制链路，步骤确认更新工作区，流程完成后触发实验总结。页面读取状态快照，避免每次刷新页面都直接访问仪器。

表 3-1 软件模块的输入与结果

软件模块	主要输入	输出或共享内容
工作区管理	指导书、学生信息、步骤确认	工作区版本、实验编号、进度与总结
语音处理	录音、引擎选择、转写文字	识别进度、操作计划与执行回执
语义规划与控制	问题、操作目标及参数	回答、操作卡及控制作业结果
后台采样与页面	仪器读回、控制完成事件	单台读数、整轮时间与变化高亮
视觉交互	最新图像、已登记人脸特征	学生身份、举手事件与步骤求助
指导书与总结	文档正文、本轮步骤及回执	校验后的指导图与本轮实验总结

3.2 从自然语言到可核对的控制作业

自然语言控制首先要把“学生想做什么”转换成可检查的操作。语音经云端或本地模型转写后，与文字输入共用同一计划入口。在线模式将近期对话、当前实验步骤和仪器状态交给 DS，并用预先定义的工具限定操作范围；离线模式解析已支持的明确指令。两种模式都先得到设备、通道和参数，再由板端检查数值范围与单位，处理流程如图 3-1 所示。

连续对话需要保留上下文，同时明确操作对象。通道选择按本句指定、此前明确指定的同一设备通道、默认 CH1 的顺序确定。例如，先说“把信号源 CH2 设为 1 kHz”，再说“幅度改成 1 Vpp”，第二句仍作用于信号源 CH2。若只修改幅度，其余设置由该通道最新回读值补齐，从而保留学生没有要求改变的参数。

计划通过校验后，控制模块创建带唯一编号的作业，记录目标设备和当前实验编号。文字入口显示操作卡，等待用户确认；语音自动模式在指令明确且校验通过时先显示操作摘要，再提交执行。同一作业重复提交时不重复操作，过期操作卡直接拒绝，同一时刻只允许一个活动控制作业。

作业按“待确认—排队—执行—成功或失败”流转。执行前先通知后台采样让出设备，再取得仪器访问锁，避免设置命令与查询命令交叉。底层程序核对设备身份、执行设置并读取实际状态；控制模块只有取得所需回读证据、确认核对通过后，才把作业标记为成功。结束时保存执行记录、触发状态刷新并释放锁，失败时同样保留原因。

语音入口还按本次语音编号去重，并区分“执行前的链路失败”与“操作已经执行后的失败”。自动模式下，DS 未连接或执行前网络不可用时，可转入本地指令解析；计划无效或仪器执行后失败时保留错误，不自动重试。这样可避免一次口令在云端和本地各执行一遍。

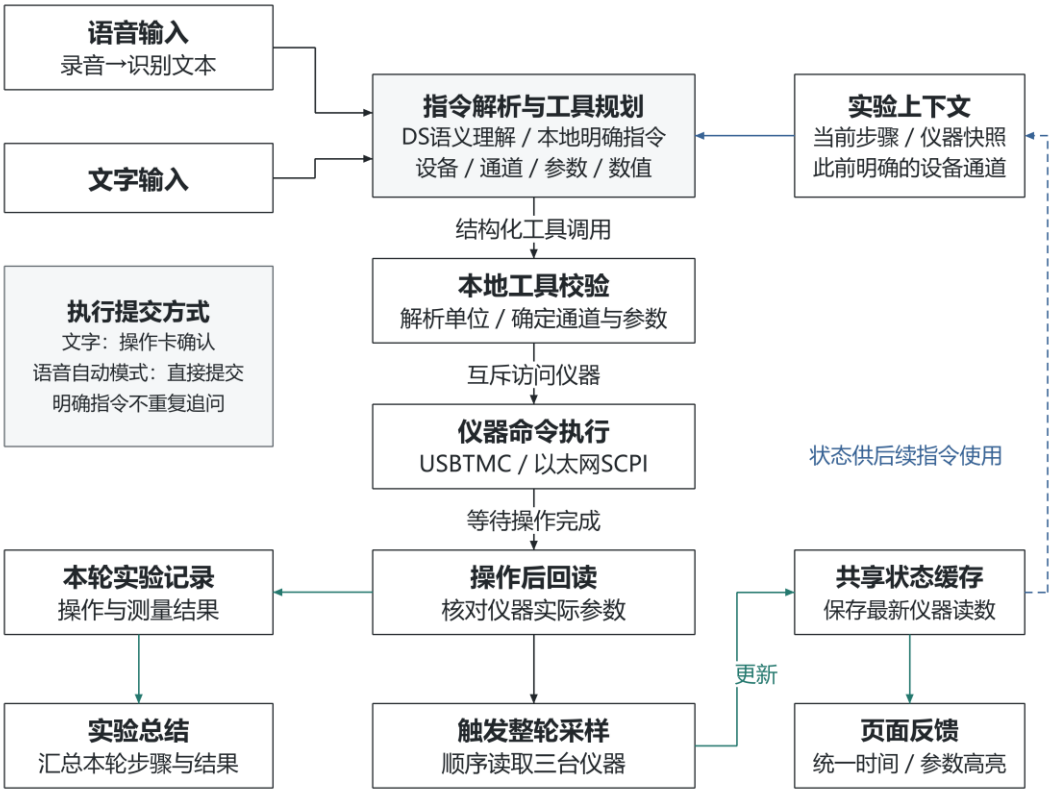


图3-1语义理解、控制作业与仪器回读闭环

3.3 仪器适配与连续配置时序

三台仪器的通信方式和命令格式不同，上层实验流程仍需要用统一方式表达操作。因此，适配层把操作整理为设备、通道、参数、单位和结果，再转换成对应型号的 SCPI 命令。SCPI 用于表达仪器设置与查询，具体命令遵循型号编程手册[4]；USBTMC 规定 USB 仪器通信方式[5]。新增仪器时，在适配层补充型号识别、命令转换和返回值解析，即可继续使用上层控制与记录流程。

DP832 电源和 AFG1062 信号源由板端 C 语言程序通过 USB/USBTMC 访问；MSO22 示波器优先通过以太网套接字访问，地址为 192.168.50.20:4000，连接后同样核对设备身份。程序采用两层互斥：进程内锁协调主服务中的控制与采样线程，目录锁协调独立运行的仪器程序。只有取得设备访问权后才允许读写，防止一项查询误读另一项操作的响应。

AFG1062 的连续波配置存在先后依赖，发送命令后必须等待仪器完成，并查询实际模式，才能继续下一步。程序先检查双通道独立模式和目标通道原输出状态，必要时暂时关闭目标输出，再准备负载、连续波模式和波形，随后设置频

率、幅度与偏置。关键步骤未确认时停止后续操作；全部配置核对通过后，按原状态恢复目标通道输出，另一通道保持原设置。

以下把 channel_control.c 中的关键判断按两个阶段展开。第一段切换连续波模式并查询确认，第二段设置正弦波并再次确认；任一步失败都立即返回错误。通道选择函数把命令中的通道模板替换为目标通道，完成检查函数则按仪器型号执行相应的等待与错误查询。

```
/* 连续波模式：发送、等待、回读 */
if (tx(selected("SOURce1:FREQuency:MODE CW")) || complete() ||
    word(selected("SOURce1:FREQuency:MODE?"), "CW"))
    return -1;

/* 正弦波形：确认后才继续设置数值 */
if (tx(selected("SOURce1:FUNCtion:SHAPe SIN")) || complete() ||
    word(selected("SOURce1:FUNCtion:SHAPe?"), "SIN"))
    return -1;
```

两类设备采用各自的完成协议：AFG1062 发送*WAI 后查询错误队列，DP832 使用*OPC?并查询错误。频率、幅度和偏置设置后，程序还读取模式、波形、负载及输出状态，按容差与目标值比较，再生成核对结果供上层形成回执。

3.4 共享缓存与三台仪器整轮刷新

仪器状态由后台统一采样，浏览器读取共享缓存。每轮依次读取电源、信号源和示波器，某台读完就立即更新该台数据；三台全部读完后，再统一发布本轮编号和完成时间。页面因此既能及时显示新参数，又能让三张卡片与底部使用相同的整轮刷新时间，流程如图 3-2 所示。

正常调度从一轮结束后等待 3 s，再开始下一轮；控制完成可提前触发，面板变化在持续检查中被发现并归入整轮更新。调度器仅在取得仪器锁、真正开始采样时确认已接收的触发事件，采样期间新到达的控制完成事件留给下一轮立即处理。较慢的读取结束后重新计算间隔，避免为追赶旧截止时间而连续补读；若控制作业正在等待，则采样及时让出设备。

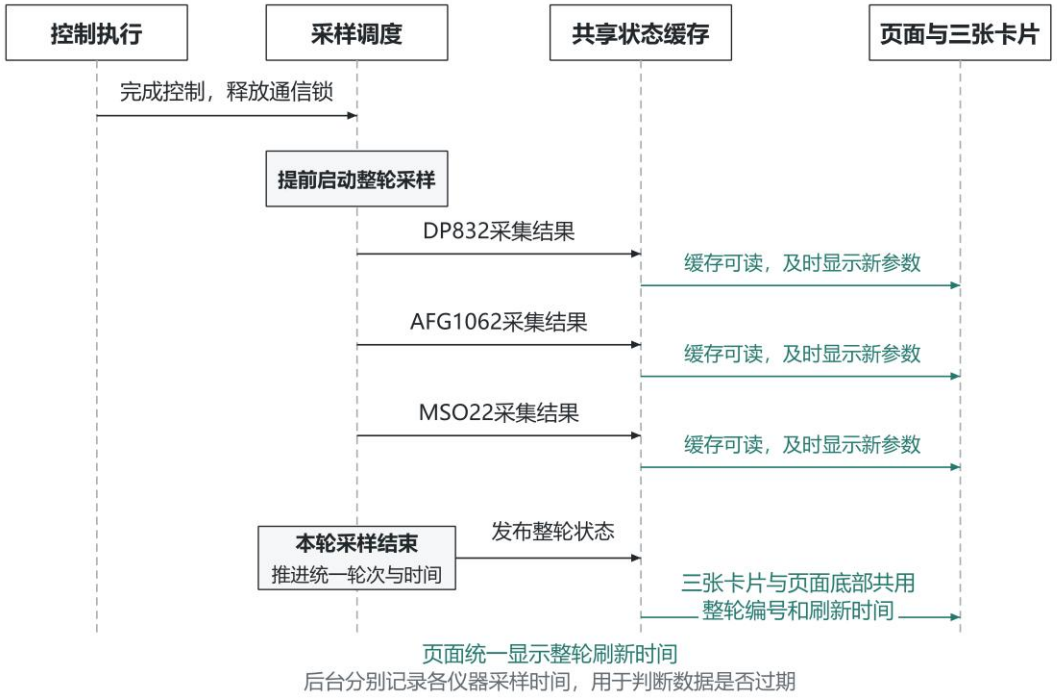


图3-2单台及时回读与整轮统一发布时间

缓存保留两种时间。单台数据年龄表示距该台最后读回经过多久，用于判断读数是否过期；整轮年龄表示距三台全部完成经过多久，用于页面统一计时。接口分别以 age 和 round_age 返回，并给三台数据附上相同的 round_revision。页面对变化参数显示约 2.2 s 的绿色高亮，三张卡片和底部共用整轮时间，同时保留对单台过期数据的判断。

3.5 语音识别与受限资源调度

语音处理包括录音、识别、计划生成和执行，页面按阶段显示进度。云端 Hy 路径先压缩录音，再经电脑转发到识别服务，输入限制为 6 s、65 kB 以内的 MP3 音频。本地路径采用基于 WenetSpeech 预训练权重的 WeNet 领域模型[6]，把 16 kHz 音频转换为 80 维声学特征，经量化编码器和 CTC 解码得到文字[7]。两条路径识别后进入同一计划入口；Hy 失败时提示用户按 BTN8 切换本地，识别引擎由用户选择。

BTN8（PB1）按键去抖后先记录用户选择，待当前资源任务结束再切换。切到本地时先暂停视觉任务、预热语音模型，加载成功后才更新当前引擎；切回 Hy 则关闭本地模型。模型加载后由常驻进程接收后续音频，以逐行 JSON 消息返回

结果，并通过单一请求锁串行处理。复用已加载权重可减少每次口令的初始化开销。

语音、视觉和文档处理共用有限内存，资源调度按任务规模区分处理。普通录音与识别先暂停视觉推理，等待摄像服务确认停止采集后再录音，任务结束时恢复视觉。加载本地语音模型前，还要求可用物理内存至少 24 MiB，且可用物理内存与空闲交换空间之和至少 192 MiB，满足条件后才允许加载。

指导书提取与生成需要更多内存，任务开始前先检查是否与录音、引擎切换、学生登记或举手求助冲突，再停止本应用所属的视觉进程组并关闭本地语音模型，实际释放模型内存。释放后可用物理内存至少 192 MiB 才接纳文档任务。结束时，按视觉进程组原内存占用加余量估算恢复预算，且不低于 256 MiB；不足时保留页面功能，提示稍后恢复识别。本地语音引擎的选择仍保留，下次请求时按需加载。



图3-3语音、视觉与指导书任务的资源交接

这两种方式分别解决“短时让出计算资源”和“释放模型内存”的问题。资源管理模块在任务开始时记录占用，在成功、取消和异常退出时都执行释放与恢复检查，避免文档处理失败后视觉长期停留在暂停状态。资源交接过程见图 3-3。

3.6 摄像、身份与举手事件协作

视觉链路需要回答两个问题：画面中是否出现有效举手，以及举手者是谁。姿态分支使用 YOLO26n-pose 输出人体框、关键点及置信度[8]，通过 NCNN 以 FP32 运行；身份分支使用 InsightFace 中的 SCRFD 500MF 和 MobileFaceNet，分别完成人脸检测与特征提取，通过 ONNX Runtime 运行。各模型职责见表 3-2。

表3-2板端视觉模型及职责

模型	用途	板端运行方式
YOLO26n-pose	人体与姿态关键点	NCNN：FP32
SCRFD 500MF	人脸检测	ONNX Runtime
MobileFaceNet	人脸特征与身份匹配	ONNX Runtime

摄像服务按需采集图像，并附上单调时钟时间戳。视觉进程从本机接口读取最新图像，丢弃采集后超过 4 s 的旧帧；人脸任务队列只保留一项待处理图像，以新帧替换积压旧帧。摄像采集与模型推理分别运行，使推理较慢时仍能获得新画面，减少使用历史动作触发求助的可能。

举手判断以肩宽作为身体尺度，降低人物远近变化的影响。先检查关键点是否可靠，再判断手腕、肩和肘之间的相对位置与前臂方向，主要条件见表 3-3。面部关键点可用时，进一步排除手贴近头部、低头触碰头部等动作；鼻子不作为举手判定的必需点。

表 3-3 举手候选的几何判定条件

判断项目	主要条件	用途
身体尺度	双肩置信度≥0.40；肩宽≥15像素	确保尺度与肩部位置可靠
手臂关键点	参与判断的肘、腕置信度≥0.45	排除低可信度坐标
手腕高度	手腕不低于同侧肩下方0.28倍肩宽	判断手部是否抬起
腕肘关系	手腕高于肘部至少0.22倍肩宽	区分上举与平伸动作
前臂方向	向上的竖直分量占前臂长度≥55%	确认前臂主要向上

单帧姿态满足条件后，系统继续积累时间证据：至少 3 个有效举手样本、首末间隔不少于 0.7 s，且举手占有效判定的比例不低于 80%，才确认事件。短暂无

法判定的帧不计票，也不立即清空已有证据；相邻有效采样间隔超过 4 s 时重新积累。触发后锁定本次动作，至少 3 个可靠放手样本持续 0.6 s 才允许再次触发；手臂局部丢失时延长放手确认至 3 s，减少遮挡导致的重复求助。

姿态分支采用两级处理：先用内部计算分辨率为 192×192 的模型持续观察，形成候选后暂停快速分支，再用原图运行输入尺寸为 320×320 的模型复核。复核同时检查人体框对应关系、人员轨迹和图像时效。身份分支异步匹配已登记的人脸特征，登记时采集三份样本；返回结果只有仍属于同一人员轨迹且未过期时才采用，避免把上一人的识别结果用于当前举手者。

复核通过后，事件携带启动标识、顺序号和唯一事件编号，避免重复轮询或进程重启造成重复处理。高分辨率复核会带来观察间隔，系统短时保留已确认事件，等待事件之后的新帧重新核对身份与轨迹，再请求当前实验步骤的建议。求助前后都检查学生与流程是否仍有效，视觉求助只提供指导信息，不直接操作仪器，协作过程如图 3-4 所示。

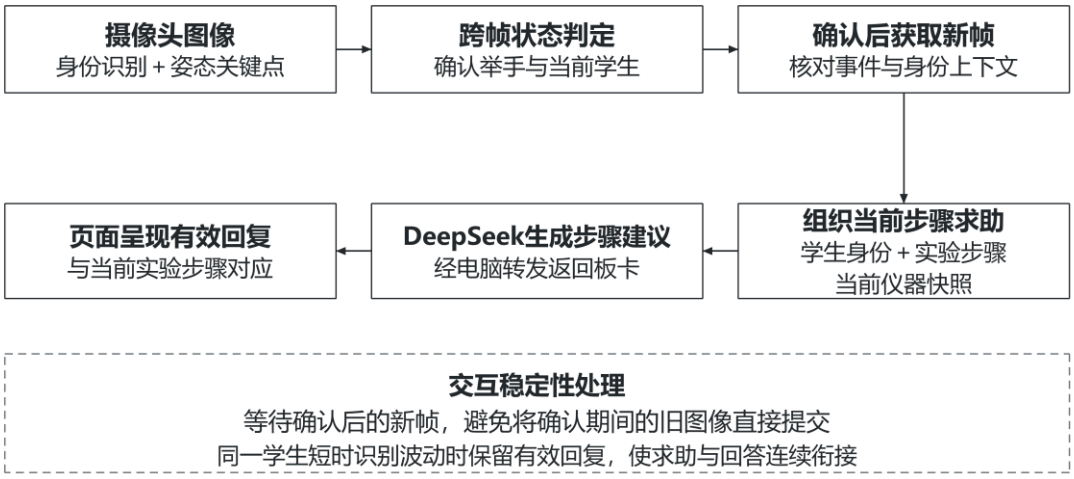


图3-4身份确认、举手复核与步骤求助

3.7 指导图生成与实验结果归属

指导书处理采用“提取—生成—校验—安装”的顺序。用户选定文件后，程序记录文件标识和工作区版本，复制时再次核对来源、大小及修改时间，防止处理过程中换入其他文件。正文由独立子进程在 25 s 期限内提取，再交给 DS 按规定模板生成节点与连线。程序核对结构及原文依据，通过后才安装新指导图；取消、提取失败、格式错误或工作区版本已变化时，均保留原指导图和实验进度。

工作区将指导书、学生、流程和总结保存到本地文件。每次开始实验都生成新的实验编号，控制作业创建时一并记录该编号。流程结束后，总结模块先筛选本轮作业，再检查是否执行成功、是否取得有效回读。以下为实验编号与回执有效性的关键判断：

```
# 只选取本轮实验的作业
if job.get("workflow_run") == run_id:
    receipt = job.get("result") or {}
    # 执行成功，并且回读核对通过
    verified = (job.get("state") == "succeeded"
                and receipt.get("verified") is True)
```

实验编号解决“结果属于哪一轮”，成功状态与回读标记解决“结果是否已确认”。在此基础上，还需区分设置回执和测量回执：示波器频率、峰峰值及游标测量等回执可作为实测依据，普通设置回执和当前缓存只说明配置状态。因此，总结请求仅携带本轮步骤、相关讨论与回执，并明确禁止把设定值写成实测值，未做的测量不补写数值。

云端总结失败时，程序使用已核对结果生成本地摘要，仍保存流程完成状态。异步总结返回后，再检查当前实验是否仍处于完成阶段、总结任务标识是否一致；若用户已重新开始实验，迟到结果不再写入，避免旧总结覆盖新一轮记录。

3.8 电脑转发与云端请求边界

云端服务通过“板卡—串口—电脑—云端”链路接入。板端把 DS 对话或 Hy识别请求提交到本机转发入口，检查访问令牌后，为请求添加唯一编号并通过 UART 发送。电脑网关核对请求字段、区分两类服务，再把云端响应按原编号返回。板端应用据此接收结果并继续本地流程，仪器控制由板端校验与执行。

电脑端转发以隐藏窗口运行，默认使用 COM15，并通过单实例约束避免多个进程争用串口。启动器确认转发已就绪后持续检查连接，断线时每 10 s 尝试重连，人工停止则退出重连循环。连接恢复后不自动补发已经超时的请求，网关也拒绝重复编号。应用仅在执行前确认链路不可用时转入离线指令规划，避免恢复连接后再次执行旧口令。

第四部分 项目完成情况、性能参数与优化验证

4.1 整机完成情况与实际成果

目前已完成三类仪器接入、统一页面、双路径语音、身份与举手求助、指导书编排及本轮实验总结。图4-1展示系统的主要组成及连接关系，表4-1说明各环节已经贯通的能力；图4-2以实拍回执核对语音要求与实际设置；图4-3展示本轮测量如何进入总结。

表4-1整机功能完成情况

验证对象	执行与观察	已验证能力
自然语言控制	语音设置示波器刻度，核对设备回读和页面高亮	指令解析、实际执行与界面反馈贯通
本地多模态	本地语音与视觉模型共存，识别期间暂停视觉，完成后恢复	模型按任务交接资源，测量查询与视觉恢复正常
上下文求助	身份显示正确，举手后返回当前步骤建议	求助关联学生身份与实验进度
多仪器状态	连续观察三张卡与底部刷新状态	整轮时间与轮次一致
自动记录与总结	本轮测量回执进入完成后的总结	测量结果自动归集，保留数值和单位

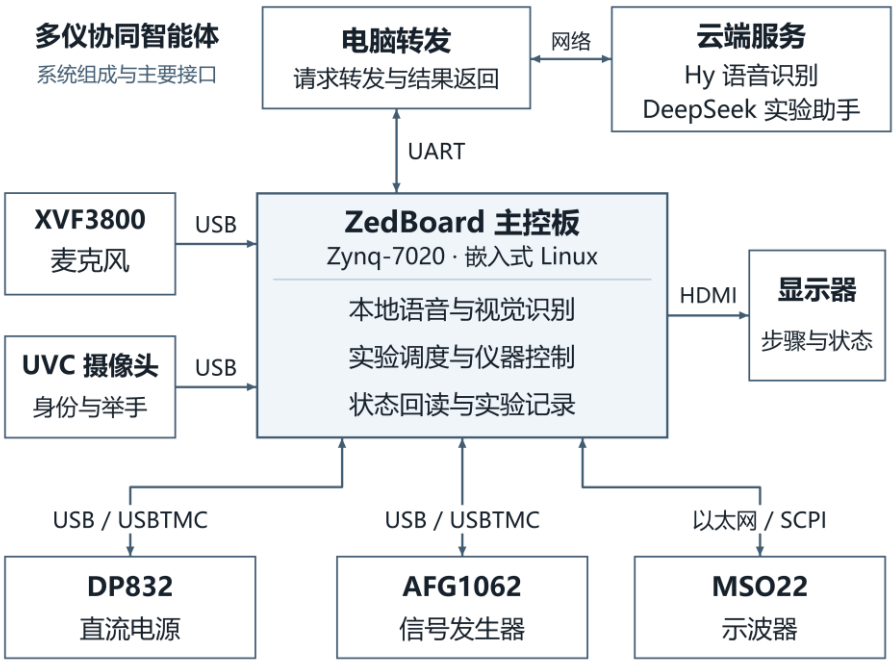
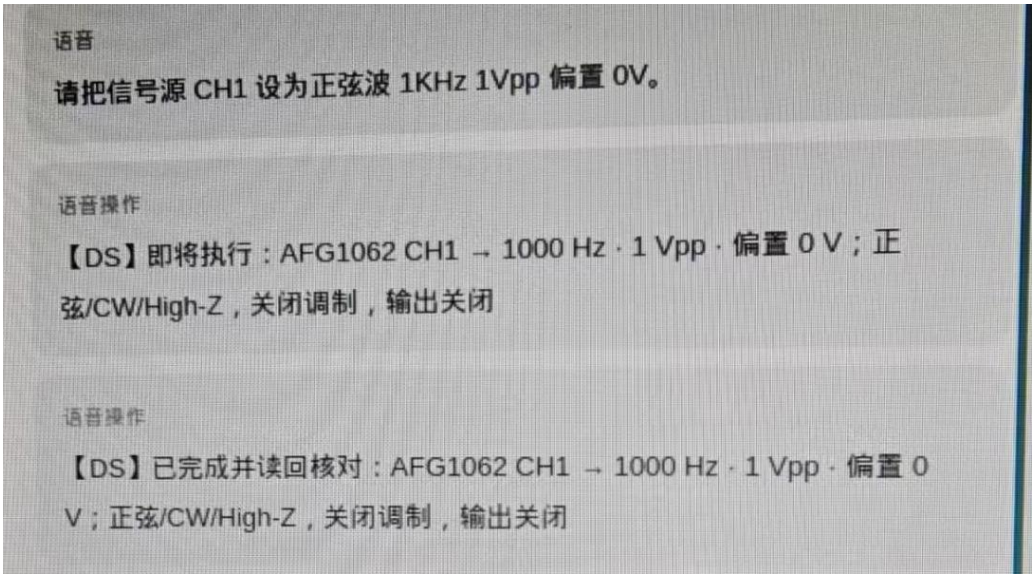
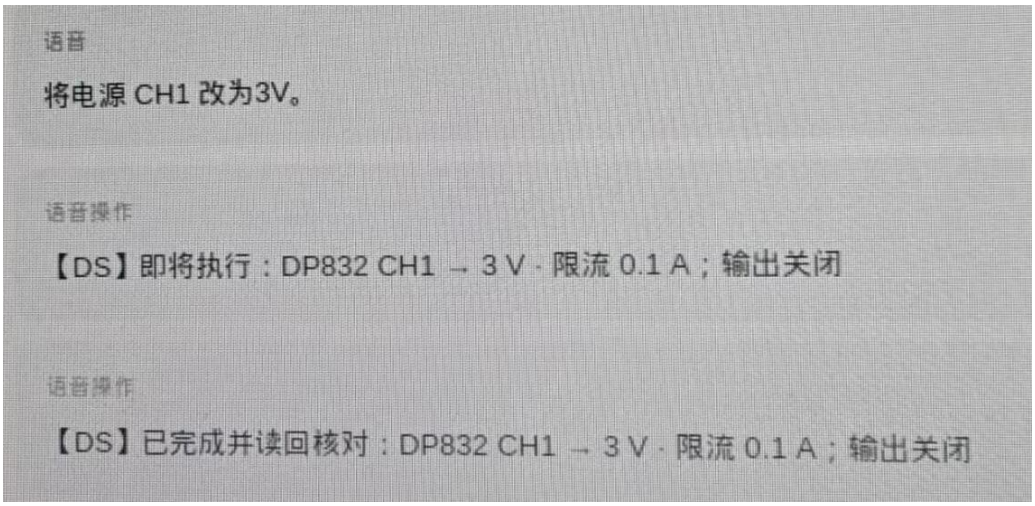


图4-1多仪协同智能体系统组成框图

图4-2选取两段实拍回执。信号源案例将CH1设为正弦波、1kHz、1Vpp、0V偏置，电源案例将CH1设为3V；回执给出实际读回设置，并显示输出均关闭。该成果展示语音、设备执行与页面反馈已连接，输出关闭状态也能在结果中核对。



（a）AFG1062：正弦波参数设置与回读



（b）DP832：CH1电压设置与回读

图4-2语音设置与仪器回读核对实拍

图4-3展示本轮实验的AI总结实拍。总结记录信号源CH1设为正弦波、1kHz、1Vpp、0V偏置，示波器时基为200μs/div，并注明相关操作已有回执确认；示波器CH1的单次采集频率为999.926Hz，换算周期约1.0001ms。该图展示参数设置与实测记录在本轮总结中的关联。

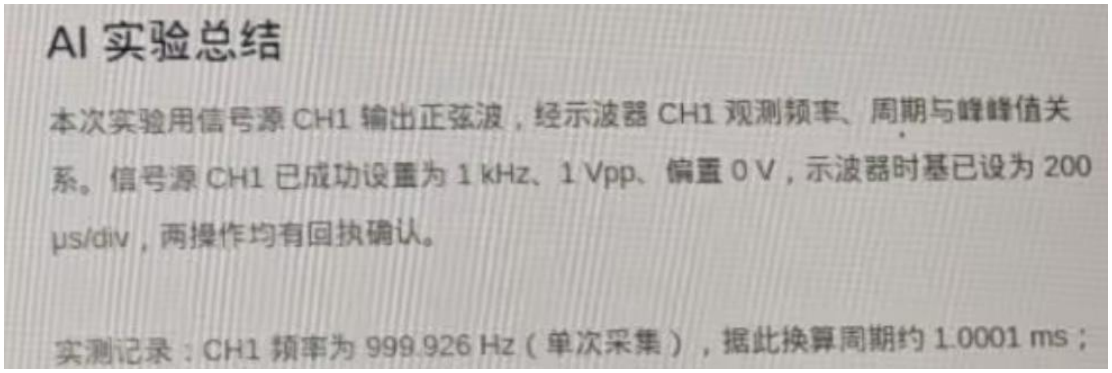


图4-3本轮实验AI总结实拍

表4-2板卡真人联调资源参数

资源指标	观察结果	记录范围
最低可用内存 MemAvailable	114372KiB，约111.69MiB	本地语音真人联调期间
最低剩余交换空间 SwapFree	151484KiB，约147.93MiB	同一测试记录
内存不足终止计数最大值	0	同一测试记录

表4-2来自sim/historical/file_44519e90a4f6.json的local-human记录，共565次采样。本轮使用本地语音，视觉、页面和仪器服务参与，识别期间按调度暂停视觉。最低MemAvailable为114372KiB，最低SwapFree为151484KiB，内存溢出计数最大值0，说明该轮执行与恢复保留了上述余量。

4.2语音模型与运行服务性能

语音评估分别回答两个问题：领域模型是否能转写仪器口令，以及保持权重不变的运行服务能否减少重复加载和程序开销。前者使用合成音频逐条核对文字，后者采用同输入对照与常驻回放，避免将模型变化和服务优化混在一起。

（1）领域模型与识别正确率

本地识别采用基于WenetSpeech语料预训练的WeNet模型[6]，经仪器领域合成语音微调后形成SmartBox V8模型。部署版本V8.2沿用该模型权重。表4-3由V8.1保存的开发集、测试集转写结果与参考文本逐条比对得到，分别考察完整录音直接输入，以及通过语音活动检测（VAD）截取说话片段后输入的效果。

表4-3本地语音模型的历史合成音频识别结果

数据集与输入	字错率CER	整句正确率
开发集原始输入（412条）	0.2803%	97.09%（400/412）
测试集原始输入（415条）	0.0644%	99.28%（412/415）
测试集VAD分段输入（415条）	0%	100%（415/415）

字错率CER为替换、删除和插入的字符错误总数除以参考文本字符数；整句正确率为转写与参考文本完全一致的句数比例。开发集原始输入有12个字符错误，参考文本共4281字；测试集原始输入有3个字符错误，参考文本共4656字。测试集截取说话片段后，415条转写均与参考文本一致。

领域语料来源记录为16156段、约10.35h：训练14046、校准1283、开发412、测试415。Qwen3-TTS合成音频由24kHz单声道转换为16kHz；开发、测试按句式文本分组，校准从原训练部分按说话人标识哈希抽取，设定比例8%。随包827条开发与测试音频、参考文本和逐样本输出支持表4-3复算。

（2）保持权重一致的运行服务优化

表4-4相同模型权重下的语音程序开销（QEMU Cortex-A9）

输入音频长度	V8.1中位耗时	V8.2中位耗时	峰值RSS前→后
1.52s	4145.60ms	4101.54ms	176040→166212KiB
2.48s	6693.62ms	6456.56ms	183360→174408KiB
4.00s	11283.80ms	10697.70ms	195348→186628KiB

表4-4来自reports/arm_benchmark.json。x86工作站通过QEMU Cortex-A9运行ARMv7程序，三条音频各重复三次；耗时取前端、编码器和解码之和的中位数，不含首次加载，RSS取每组三次峰值的最大值。V8.2耗时分别下降1.1%、3.5%、5.2%，RSS下降5.6%、4.9%、4.5%。该比较反映模拟环境中的程序开销，板卡共存参数另见表4-2。

（3）常驻服务的复用与输出一致性

语音运行包中的service_validation.json记录了在x86工作站上进行的回放验证。测试使用开发集与测试集合计827条历史合成音频，分别输入原始音频和经过语音活动检测（VAD）分段的音频，共完成1654次推理。整个回放过程仅加载

模型一次，三个独立客户端也复用了同一服务进程，说明模型可以跨请求、跨客户端持续复用。

上述1654次回放中，V8.2的识别与指令解析输出均与V8.1对应基线一致，不一致数0。它验证了常驻改造的输出一致性和一次加载、多次复用。

4.3视觉推理与举手事件性能

视觉优化针对持续姿态观察的计算量与短暂关键点失效。公开图像评价手臂关键点定位，P01开发录像评价完整举手事件，两类输入和结果分别保存在data/evaluation/vision/coco与p01。

（1）运行实现与分辨率取舍

表4-5视觉耗时与定位质量的两组评估

评估项目	**方案**及结果A	**方案**及结果B
运行耗时对比：推理实现	ONNX Runtime，320计算	NCNN，内部192计算
QEMU重复推理中位耗时	3607.67ms	786.47ms
独立分辨率评估：计算规格	quality 320	fast 192
手臂PCK（误差≤0.20倍肩宽）	71.81%	63.83%

表4-5耗时来自fast_validation.json。在QEMU Cortex-A9中，两种实现接收同一帧生成的1×3×320×320 FP32张量，各运行4次，舍弃首次后取3次中位数。ORT 320改为NCNN内部192计算后耗时降低78.2%。该结果同时改变运行库和计算分辨率，反映组合优化。

分辨率评估来自public_arm_comparison.json，使用电脑端ORT CPU推理。从900张COCO验证图像中得到421张上半身裁剪图，计分458人、1894个可见手臂关节。人体框按IoU≥0.5匹配，标注肩宽至少15像素；肩、肘、腕共6类关节中，可见性为2的关节参与统计。预测置信度≥0.45且位置误差≤0.20倍标注肩宽计为正确，漏检计错。320与192计算分别得到1360/1894（71.81%）和1209/1894（63.83%）的PCK。

192计算更快，但手臂定位PCK低于320，故采用3.6节的快速候选与320复核。两分支共用320×320输入及坐标接口，快速分支在网络内部缩小图像并将输出映射回320尺度；导出通过pnnx生成NCNN结构与权重[9]。这一实现把持续观察开销和关键动作确认分开，保留质量较高的最终复核。

（2）多帧举手事件与容错

初版规则把鼻子关键点作为必要条件，并要求动作严格逐帧连续。开发录像中，鼻子置信度不足或短暂关键点失效会中断举手判定。改进后使用肩—肘—腕的相对位置判断手臂状态，结合多帧投票与短暂失效容忍，将连续动作归并为一次求助事件。

表4-6举手规则改进前后的开发录像回放

回放内容	V1触发次数	V2触发次数
右手举手2次	0	2
左手举手2次	0	2
非求助片段约80s	0	0

回放使用同一参与者P01、同一机位的3段开发录像，共1638帧、约221s。这些录像参与规则调试：V2检出4次标注举手，约80s非求助片段触发0次；采用快速筛选与320复核后，事件结果一致。对保存关键点施加鼻子置信度置零、每5帧将1帧设为无效、左右镜像等扰动，仍检出4次举手，未增加误触发。此项验证了开发样本上的事件判定和容错能力。

4.4仪器与交互改进效果

表4-7仪器与交互问题的改进对照

观察到的问题	改进方式	验证结果
控制后页面仍等待周期采样	控制完成事件触发整轮刷新	三张卡片与页面底部轮次一致，整轮时间同步更新
要求正弦波，实际读回方波	模式、波形逐项等待并回读	三组CH1方波转正弦波检查通过
举手确认时使用了较早图像	等待新帧并保留短时身份波动中的回复	真人举手取得当前步骤DS建议
总结可能混入上一轮记录	按实验编号筛选有效回执	实际DS流程验证重新开始与旧总结清除

AFG1062修复前后采用相同配置操作对照。旧程序要求正弦波（SIN），实际仍回读方波（SQU）；单独发送波形指令并等待后能正确切换，故将问题定位到连续配置的执行顺序。修正后逐项完成负载、模式和波形设置，再设置频率、幅度与偏置。skill/evidence/candidate-hardware.json保存三组CH1实机记录：原输出关闭、目标2Vpp；原输出开启、目标1Vpp；原输出开启、目标2Vpp。三组波形与幅度均回读正确，操作前后的输出状态一致；测试结束恢复原设置，CH2保持不变。

4.5验证范围与结果复核

板卡验证分批保留原始记录。2026年10月28日的38项模型检查覆盖三轮本地语音与Hy切换、约五分钟模型共存；23项DS综合检查覆盖Word/PDF指导书、问答、进度、总结与通道上下文。10月30日的同步修复另有10项原生控制、5项页面结果处理和10项DS就绪检查，以及三组AFG1062实机案例。各批次对应skill/evidence/ 中 的 model-checkpoints.json 、 ds-integration.json 与 local-test-results.json等记录。

历史电脑端sim/results/offline_checks.json记录51个应用用例：50个通过，1个PDF用例因缺pdftotext跳过；后者在另配Poppler 24.04.0环境通过并保存于pdf_check.json。11月2日的模型评测包含语音保存输出复算、公开图像ORT CPU重新推理及P01关键点规则回放，方法见sim/model_evaluation.md。

2026 年 11 月 3 日的离线复核在 Windows 临时副本重跑刷新 5 项、回执 7 项、重连 4 项、就绪 11 项，共 27 项均通过；评测输入及保存输出核验 21 项、材料统计和 ADI 参考归档核验 18 项也通过，9 份技能证据摘要相符。此轮核对当前交付代码的离线逻辑、已保存结果与输入完整性。

综上，板卡实拍及回执说明整机功能完成情况，QEMU 用于相同条件下的 ARM 程序对比，x86 回放用于常驻一致性，公开图像推理与开发录像分别评估定位和事件。复算脚本检查统计是否自洽，归档校验确认材料未变。真实多人、复杂环境和长时间连续运行的量化评估列为后续工作。

第五部分 开发协作与工程复现

5.1协作方式与人工核对

开发阶段使用Codex分析现象、源码和日志，按“提供原始问题—提出原因—最小复现—局部修改—核对回读与页面”的过程迭代，人工负责纠正设备对象、核对物理状态和验收结果。本节记录开发协作中的真实提示、模型回答及被测试修正的判断。

5.2真实提示、回答与纠错过程

信号源波形切换问题体现了从用户反馈到代码修正的协作过程。skill/collaboration.md整理了2026年10月30日的原始交互，skill/evidence/quotes.json保存摘录及对应对话、轮次标识。以下按问题出现、定位与验证的顺序说明。

用户首先反馈“执行错误”。助手回答：“DS 返回成功只说明请求有回复，不能证明执行正确。”随后用户进一步指出是信号源，将排查对象明确为AFG1062，助手据此纠正范围。检查依次对齐语音文本、DS解析参数、发出的命令与仪器回读，发现目标波形是SIN，执行后的实际状态却是SQU，问题出现在设备配置阶段。

对照实验进一步缩小了原因。助手记录：“已复现同样的故障：旧程序发送正弦波设置后，仪器仍返回方波；单独发送同一条设置并等待完成，则能正确切换。”因此在模式、波形切换后增加完成等待、错误查询和状态核对，确认生效后才继续后续参数设置。

修正后分别检查原输出关闭及开启的三组配置，助手反馈：“三组实机测试全部通过：都正确从方波切到正弦波，幅度回读正确，原先开启的输出也成功恢复。”旧失败、修正后的回读与测试摘要分别保存在old-reproduction.json、candidate-hardware.json和local-test-results.json，三者对应同一问题的发现、修改与验收。

该案例将“回复成功”和“设备生效”分开。运行提示ds_prompt.py也写入“工具返回计划不等于仪器已完成；执行结果以本机回执为准”。原始对话、模型回答、工程改动与三组实机记录相互对应，作为后续技能S1的来源。

资源调度的纠错始于长录音。2026年10月23日，用户希望语音与视觉常驻以减少等待，联调却出现“这次长录音识别时内存耗尽，系统关掉了显示界面的浏览器进程”。用户提出限制最大录音时长后，程序加入采集时长与数据完整性检查，并调整视觉暂停、恢复流程。后续记录为“连续两次 6 秒测试已通过：没有新增内存崩溃，界面正常，摄像头都已恢复。”最终实现还在模式切换和指导书处理时管理模型加载、释放与恢复。

实验总结曾出现后台已生成、页面仍看不到的情况。用户反馈“看不到”后，助手发现板卡浏览器停留在更新前的旧页面。这促使验收同时检查持久化记录与实际页面。归档时，又将有999.957Hz、1.0752V测量回执的真人测试轮，与无测量回执的新一轮分开保存；程序通过实验编号筛选回执，并以总结任务标识隔离重开实验后迟到的结果。

电脑转发曾在一次连接成功后退出，重新插线也未恢复通信。助手根据进程和启动配置发现，后台任务缺少登录启动及退出后的重连处理。修正后增加延迟重连、人工停止与当前启动段就绪判断。连接恢复后，重复测试均无断联情况。

5.3技能提炼、组织与验证

从上述具体纠错提炼五项可复用提示工作流，均包含适用范围、必要输入、排查顺序、输出、验收方式和失效条件。技能来自实际工程。S1与S3依赖历史板卡记录，S2、S4、S5另有可直接重跑的离线测试。

5.3.1仪器命令完成与参数回读

适用于具有状态查询接口的仪器或设备控制。输入设备型号、通道、目标参数、初始输出状态及按时间排列的命令与回读；先找目标与实际状态的第一个不符点，再检查完成等待、错误查询和参数设置顺序。验收同时比较目标通道回读、原输出状态及其他通道状态。该方法由AFG1062的SIN/SQU故障提炼，修正效果由三组实机记录支持。迁移到其他协议时，应替换其完成信号与查询方式。

5.3.2多仪器整轮刷新

适用于多设备共享一次刷新轮次的界面。输入设备清单、轮询周期、读取耗时、变化事件及页面字段，明确单台读数更新与整轮结束的先后关系：新读数及

时发布，全部设备完成后更新共享轮次，下一周期从完成时刻计起。5项模拟时钟测试覆盖忙时事件、读取中变化和慢读；历史界面检查核对绿色高亮及三卡、底部时间一致。单台数据年龄继续用于过期判断。

5.3.3受限内存下的模型调度

适用于多个模型共享有限内存的嵌入式应用。输入各模型峰值占用、任务状态、资源所有者及失败日志，区分暂停计算与卸载模型，并逐一检查成功、取消、异常路径的释放和恢复。多仪协同智能体据此形成按需加载本地语音、切回Hy时卸载、文档处理时释放模型的安排，38项模型检查及23项综合检查覆盖其切换与恢复。换模型、输入长度或板卡后，需要重新测量峰值并确定准入阈值。

5.3.4实验回执归属与异步结果隔离

适用于按任务或实验批次生成总结的应用。输入当前实验编号、作业所属轮次、成功状态、验证标记与实际回读，按轮次选取结果，并将测量回执、设置操作与当前设置分开。异步写回时还要核对流程完成状态和总结任务标识，防止旧结果覆盖新实验。该技能结合页面未显示总结的排查和不同实验轮次的归档形成，7项模拟测试检查归属、重启、重复请求、迟到响应与云端失败等情况。

5.3.5转发启动就绪与断线重连

适用于串口桥接、后台代理等需要持续连接的程序。输入进程、任务状态和本次启动日志，联合检查目标路径、唯一进程、PID与最新ready记录；断线后延迟重试，人工停止时退出。该技能来自“曾经连接成功，后来进程退出”的故障，4项模拟重连测试、11项就绪判定案例及三次真实DS回复分别验证恢复逻辑、状态判断和链路。

五项技能已经位于完整工程的labmate/skill/中，无需再解压单独技能ZIP。README.md说明选择与应用，prompts.md给出可填写模板，pitfalls.md列出排查清单，engineering.md关联代码，checks.md说明验证，collaboration.md与evidence/保存真实来源。复用时先替换自己工程的设备协议、状态字段、任务标识和资源条件，再运行目标工程的验收。

verify_evidence.py核对9份归档文件的摘要与结果字段；run_checks.py把当前相关源码及测试复制到临时目录，重跑模拟时钟、工作区和转发替身测试。11月1日与2日的27项记录保留原日期，11月3日针对当前完整工程的重跑仍为27项通过。

例如复用整轮刷新技能时，先填写参与设备、周期、读取耗时和事件类型并提供调度及页面代码。模拟案例以100.0s开始、101.2s完成，周期3s，预期104.19s不触发、104.2s触发，用于检查周期从完成时刻起算。再用下面入口重跑5项调度测试，最后按checks.md核对真实界面高亮、统一计时和过期恢复。

```
python -B skill/run_checks.py --project-root . \
  --only refresh --output ../refresh-check.json
```

5.4复现材料与环境

交付以完整labmate/目录组织，含源码、模型、运行程序、部署工具、测试输入和skill/。本章相对路径均以labmate/为起点；先阅读根目录README.md，保留目录结构及pc-relay与pc-background-launcher的相邻关系。采用当前交付载荷恢复运行目录，不用旧版本覆盖已有工程。

表5-1复现所需材料与入口

目录	内容与使用入口
src	boot/labmate-control-v2包含主应用、语音V8.2和视觉V2；另有辅助应用。pc-relay与pc-background-launcher须保持相邻，用于电脑转发。
board	system_setup.md、README.md说明系统准备与运行；两份manifest记录运行路径和权限；config提供配置说明，adi_reference保存显示启动文件的来源资料。
build	restore_runtime.py恢复原运行目录，install.sh安装到新系统；toolchain.md和model_reproduction.md说明源码构建及依赖。
sim、data	sim包含验证脚本、历史记录和results结果；data/evaluation提供语音回放音频、公开图像与P01关键点等评测输入。
skill	五项提示工作流、踩坑清单、协作记录与证据；verify_evidence.py检查归档，run_checks.py运行离线功能测试。

硬件采用ZedBoard、SD卡、显示器、DP832、AFG1062、MSO22、XVF3800麦克风、UVC摄像头、USB集线器及BNC线。系统为ADI Kuiper GNU/Linux 11.2（bullseye），ARMv7，内核5.10.0-98248-g1bbe32fa5182，Python 3.9.2。归档环境

包含Chromium 101.0.4951.57、OpenCV 4.5.1、NumPy 1.19.5、pyserial 3.5和Poppler 20.09.0，PDF正文提取使用pdftotext。板端模型、可执行文件与配套库按原件恢复。

电脑转发需Python 3.9+；评测输入核验需Python 3.12+与Pillow，全部技能测试还需Windows PowerShell。源码重建使用x86-64 Ubuntu 24.04、GCC 13.3.0 arm-linux-gnueabihf及配套sysroot，具体版本见build/toolchain.md。配套显示启动文件来自ADI的ZedBoard ADV7511参考设计，原产物由Vivado 2021.1生成；来源与校验信息保存在board/adi_reference。

5.5从零准备与部署步骤

（1）检查材料。在labmate/根目录执行以下命令，结果输出到目录外。restore_runtime.py未指定目标目录时只校验载荷，其余命令分别核对保存统计、评测输入、技能归档及离线逻辑。技能全组需要Windows PowerShell；Linux使用python3并按--only选择Python测试组。

```
python -B build/restore_runtime.py --scope all
python -B build/verify_materials.py --output ../material.json
python -B sim/verify_evaluation_data.py --output ../input.json
python -B skill/verify_evidence.py
python -B skill/run_checks.py --project-root . --output ../skill.json
```

应用回归另执行python -B sim/verify_offline.py --output ../offline.json。该入口在临时目录检查路径恢复、语法和应用逻辑；JavaScript语法检查需要Node.js，PDF用例的后端条件见sim/README.md。检查输出中的PASS与SKIP项目后，再进入板卡部署。

（2）准备基础系统SD卡。按board/system_setup.md从ADI历史发行页[10]选择2022-08-04的2021_R1完整镜像image_2022-08-04-ADI-Kuiper-full.zip，核对MD5：9201b9e6580a0ce5c606f40f99c11b9a。解压为IMG，在Balena Etcher中选

择镜像和至少16GB的目标SD卡，确认目标卡后写入并等待校验[11]。该操作会替换目标卡内容，应先保存卡中数据。

（3）恢复应用。在Linux工作站进入labmate/，将载荷恢复到尚不存在的暂存目录。脚本按board中的两份manifest还原Linux路径、大小写、权限、属主及符号链接，模型与程序从恢复目录安装使用。

```
sudo python3 -B build/restore_runtime.py --scope all \
  --root /tmp/labmate-stage-new
```

（4）安装新卡。将新卡rootfs挂到/mnt/new-root，BOOT分区挂到/mnt/new-root/boot，执行以下命令。安装脚本复制三个应用目录、服务和页面切换入口，遇到已有应用会停止；四个启动文件来自配套ADI参考归档，保留其版本对应关系。

```
sudo sh build/install.sh /mnt/new-root /tmp/labmate-stage-new
sudo cp -a /tmp/labmate-stage-new/boot/BOOT.BIN \
  /tmp/labmate-stage-new/boot/devicetree.dtb \
  /tmp/labmate-stage-new/boot/uImage \
  /tmp/labmate-stage-new/boot/uEnv.txt /mnt/new-root/boot/
```

然后设置桌面入口和 256MiB 交换文件。以下命令仅用于离线挂载的新卡；交换文件须尚不存在。systemd 单元名由路径生成，保留新卡自身 fstab 及分区标识。

```
sudo install -d -o 1000 -g 1000 \
  /mnt/new-root/home/analog/.config/autostart
sudo cp -a /tmp/labmate-stage-new/home/analog/.config/autostart/labmate-
display.desktop \
  /mnt/new-root/home/analog/.config/autostart/
test ! -e /mnt/new-root/var/labmate-model.swap || exit 1
sudo dd if=/dev/zero of=/mnt/new-root/var/labmate-model.swap \
  bs=1M count=256 status=progress
sudo chmod 600 /mnt/new-root/var/labmate-model.swap
sudo mkswap /mnt/new-root/var/labmate-model.swap
swap_unit=$(systemd-escape --path --suffix=swap /var/labmate-model.swap)
sudo install -m 644 board/rootfs/etc/systemd/system/file_dcb5d7aae3d0.swap \
  "/mnt/new-root/etc/systemd/system/$swap_unit"
sudo systemctl --root=/mnt/new-root enable "$swap_unit"
sudo systemctl --root=/mnt/new-root enable labmate-camera.service \
  labmate-display.service labmate-buttons.service
```

（5）填写配置。主服务也需要/etc/labmate/pc-relay.env，即使本地模式也先建立。按下面命令为新卡生成独立 token；文件已有时停止。DS 和 Hy 各自的 API key 由使用者填写，不能互用；文件属主 root、权限 0600，具体文件路径见代码。

```
sudo python3 - /mnt/new-root <<'PY'
import os, secrets, sys
from pathlib import Path
root = Path(sys.argv[1]).resolve()
assert root != Path('/') and (root/'etc/os-release').exists()
folder = root/'etc/labmate'
folder.mkdir(mode=0o700, parents=True, exist_ok=True)
fd = os.open(str(folder/'pc-relay.env'),
             os.O_WRONLY | os.O_CREAT | os.O_EXCL, 0o600)
with os.fdopen(fd, 'w') as stream:
    stream.write('LABMATE_CLOUD_RELAY=serial\n'
                 'LABMATE_RELAY_TOKEN=' + secrets.token_hex(24) + '\n'
                 'DEEPSEEK_MODEL=deepseek-flash\n')
PY
sudo touch /mnt/new-root/etc/labmate/deepseek.key \
  /mnt/new-root/etc/labmate/hy-asr.key
sudo chown root:root /mnt/new-root/etc/labmate/*.key
sudo chmod 600 /mnt/new-root/etc/labmate/*.key
sudoedit /mnt/new-root/etc/labmate/deepseek.key \
  /mnt/new-root/etc/labmate/hy-asr.key
```

安全卸载后，在断电板卡插入 SD 卡并选择 SD 启动。用 115200、8N1、无流控连接 UART，核对 uname -a、/etc/os-release 和 python3 --version；用 swapon --show --bytes 及 free -k 确认交换空间。历史总 Swap 为 364536KiB，包含系统原有部分，应用新增文件为 256MiB。默认语音引擎 Hy，按 BTN8 / PB1 选择本地后不需要 ASR 云凭据。

按board/system_setup.md核对python3-pil（Pillow）、python3-numpy、alsa-utils、ffmpeg、lame和poppler-utils，缺少时从对应发行版包源安装。Pillow与NumPy供视觉处理使用，ALSA供录音使用，FFmpeg与LAME处理音频，pdftotext提取PDF正文；通过导入PIL、numpy及检查arecord、ffmpeg、lame、pdftotext命令确认依赖可用。

（6）连接硬件。按图2-2连接三台仪器、麦克风、摄像头、USB集线器与显示器。板卡eth0为192.168.50.10/24，MSO22为192.168.50.20并启用4000端口Socket Server。DP832、AFG1062的USB DEVICE口接板卡主机通路，核对型号匹配。BNC信号接线与通信线分别核对，初次启动先保持仪器输出关闭。

（7）启动板端。在UART终端执行以下命令并检查三个服务为active。labmate-camera.service是主界面视觉取帧的前置服务；若模型或采集异常，分别查看摄像

服务日志与journalctl -u labmate-display.service，不要只以页面可打开判断识别正常。首次使用由应用创建工作区，学生身份需要在页面登记。

```
sudo systemctl daemon-reload
sudo systemctl enable --now labmate-camera.service
sudo systemctl start labmate-display.service labmate-buttons.service
systemctl is-active labmate-camera.service \
  labmate-display.service labmate-buttons.service
```

（8）启动电脑转发。UART终端先登录板卡root shell，确认id -u返回0，再断开终端释放串口。电脑使用Python 3.9+及同目录pythonw.exe；保留相邻的src/pc-relay和src/pc-background-launcher。进入src/pc-relay后，在PowerShell执行下列命令，将Python路径换为本机实际路径。

```
powershell -NoProfile -ExecutionPolicy Bypass `
-File .\start-cloud.ps1 -Port COM15 `
-Python C:\Python39\python.exe
```

默认COM15启动方式会配置电脑登录自启动并保持后台运行；其他端口可显式使用-Foreground前台启动。连接失败每10s重试，先前失败的请求由用户重新发起。检查labmate/logs/pc-relay/cloud.log中当前进程本次启动段的UART relay ready，并确认进程持续运行。维护前执行stop-cloud.ps1，等待退出并释放串口；请求进行中可能需约45s。维护结束后按同一入口恢复单个转发进程。

在板卡浏览器打开http://127.0.0.1:8765，核对三台仪器状态后，按2.4节完成整轮实验，并以表5-2验收。需要UART维护时先按转发停止流程释放串口，维护结束再恢复单个转发进程。现有工作区display-data/workspace.json包含学生与实验进度，更新应用时应保留。

表5-2完整使用流程的复现验收点

检查点	预期结果	异常时优先检查
指导书生成	步骤与指导图成功显示	文件解析、DS链路、可用内存
语音设置	设备回读与页面数值一致	设备通道、单位、工具回执
整轮刷新	三卡与底部同轮次；变化有高亮	控制完成事件、采样锁、缓存
举手求助	对应学生收到当前步骤建议	身份确认、新帧与事件状态
测量与总结	本轮回执中的结果进入总结	实验标识、测量回执、页面版本

遇到问题时，先沿“输入—解析—执行—回读—页面—总结”查找第一个不一致的位置。语音问题分别检查转写文本与解析参数；设备问题核对命令、错误查询和实际回读；总结问题核对实验编号与成功回执。这样可以区分语音识别、云端请求、仪器执行和页面显示各环节的故障。

5.6 源码重建与评测复现入口

常规部署使用配套ARM运行程序。需从源码重建时，按build/toolchain.md准备固定版本工具链，设置TOOLCHAIN_ROOT，再在恢复后的可写主应用目录执行sh build.sh与sh build-channels.sh。以下check_linux.py入口在临时目录验证恢复、离线安装和六个仪器C程序编译；状态读取与辅助摄像程序分别使用build/build_status.sh、build/build_camera.sh，参数见build/README.md。

```
python3 -B build/check_linux.py \
  --toolchain /path/to/toolchain --output ../linux-check.json
```

重建语音与视觉程序时，在labmate/执行python3 -B build/prepare_model_workspace.py --runtime-root /tmp/labmate-stage-new --output /absolute/new-build-dir，输出目录须不存在。脚本创建可写副本并适配开发路径；随后按build/model_reproduction.md准备固定运行库和ORT补丁，运行语音prepare_exact.py、build_probe.py、build_service.py及视觉构建入口。编译使用随包模型，重新导出所需训练权重按该文档另行准备。

模型评测按sim/model_evaluation.md进行：先校验输入，再复算保存输出、回放关键点或重新推理。新结果另存，使用相同样本和指标对比，标明运行环境与验证层次。

第六部分 总结与展望

6.1作品总结

多仪协同智能体将ZedBoard、三台仪器与语音、视觉、页面交互集成为电子实验工作台。学生可导入指导书、查看步骤，通过语音调整仪器或读取测量值，提问或举手获取帮助，结束后查看本轮总结。Hy与本地模型分别提供云端、板端语音识别，DS结合当前实验给出指导，板端负责设备执行、回读和记录。

系统重点解决了功能衔接中的问题：信号源配置逐段等待并核对状态；新参数及时显示，三台仪器统一整轮计时；模型按任务复用、暂停或卸载，减少重复加载并控制内存峰值；操作、测量和总结按实验编号关联，使结果能追溯到实际回执。

验证从模型、程序到整机逐层展开。827条合成音频形成的1654次回放中，V8.2与对应基线输出一致，模型仅加载一次；QEMU对比记录程序开销变化；板卡联调完成测量、求助、总结与资源恢复，保留了回执和内存采样。开发中的仪器同步、整轮刷新、模型调度、实验归属与转发恢复经验整理为五项技能，配套真实案例、提示模板和检查入口。

6.2扩展方向

首先完善真实使用条件下的评测。语音增加不同说话人、口音和实验室噪声录音，分别统计转写、解析与整次操作的成功率；视觉增加使用者、机位、光照与遮挡条件，按举手事件统计漏报、误报和响应时间。保留独立测试样本，将参数调整与最终评估分开。

其次开展连续多轮板卡实验，分环节记录语音转写、模型推理、仪器查询和云端请求耗时，统计内存峰值、异常与恢复情况。根据主要等待环节调整调度与交互反馈，检验断网、串口重连及模型恢复后能否继续当前实验。

应用扩展沿现有接口增加仪器适配，为不同课程建立实验模板；利用按轮次保存的步骤、回执和测量数据，增加标准实验报告导出并保留数据来源。技能则在新的设备或任务场景中记录适配与验证过程，形成独立于本项目的复用案例。

参考文献

[1] NI．NI-VISA[EB/OL]．[2026-11-02]。

[2] Keysight Technologies．BenchVue Data Manager View[EB/OL]．[2026-11-02]。

[3] AMD．Zynq-7000 SoC Technical Reference Manual（UG585）[EB/OL]．Version 1.15，2026[2026-11-02]。

[4] Tektronix．AFG1000 Series Arbitrary Function Generator Programmer Manual[EB/OL]．077-1129-01[2026-11-02]。

[5] USB Implementers Forum．Universal Serial Bus Test and Measurement Class Specification（USBTMC）[EB/OL]．Revision 1.0，2003[2026-11-02]。

[6] ZHANG B, LV H, GUO P, et al．WenetSpeech: A 10000+ Hours Multi-domain Mandarin Corpus for Speech Recognition[C]//ICASSP 2022．2022: 6182-6186。

[7] GRAVES A, FERNÁNDEZ S, GOMEZ F, et al．Connectionist Temporal Classification: Labelling Unsegmented Sequence Data with Recurrent Neural Networks[C]//Proceedings of the 23rd International Conference on Machine Learning．2006: 369-376。

[8] Ultralytics．Pose Estimation with Ultralytics YOLO[EB/OL]．[2026-11-02]。

[9] Tencent．ncnn: A High-Performance Neural Network Inference Framework[EB/OL]．[2026-11-02]。

[10] Analog Devices．Kuiper Linux Release Notes[EB/OL]．2022-08-04（2021_R1）历史发行项[2026-11-02]。

[11] Analog Devices．Using Kuiper Images[EB/OL]．Writing the Image to an SD Card[2026-11-02]。